Denial of Service Security Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ Denial of Service (DoS) — Defensive Security Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Availability testing can disrupt services. Use staging, local labs, or explicit written authorization with agreed traffic and payload limits. Do not run floods, oversized requests, or resource-exhaustion tests against production without a formal plan.*

■ Executive Summary

Denial of Service (DoS) occurs when requests or inputs consume enough resources to prevent a service from operating normally. The supplied three-page PDF lists several potential resource-exhaustion areas, including oversized cookies and inputs, image processing, regular expressions, compression, HTTP headers, and request-limit behavior.

■ Core Testing Methodology

1. **Agree on a test plan.** Define scope, time window, request budget, monitoring, emergency contact, and stop conditions.
2. **Map resource-intensive features.** Review uploads, image resizing, search, parsing, registration, and endpoints that accept large inputs.
3. **Establish a baseline.** Measure normal latency, error rate, CPU/memory use, and response size in staging.
4. **Test input limits safely.** Use incremental, bounded test values and stop before resource use becomes abnormal.
5. **Review cookie and header limits.** Confirm oversized cookies and headers are rejected with safe client errors.
6. **Review image processing.** Enforce file size, pixel count, dimensions, decoding time, and memory limits.
7. **Review regex and parsing.** Inspect expensive regular expressions and parsers for pathological worst-case behavior.
8. **Review compression and caching.** Check safe decompression limits and ensure cache/proxy layers agree on request interpretation.
9. **Review rate limiting.** Confirm documented limits are effective without attempting a high-volume flood.
10. **Document and retest.** Record metrics, exact safe test bounds, and the observed rejection behavior.

■ High-Value Bug Bounty Tips & Edge Cases

- **Cookie bomb:** Oversized cookies can burden request parsing and proxy infrastructure.
- **Long form values:** Long passwords, email fields, and other inputs should be bounded before expensive processing.
- **Image dimensions:** Large pixel dimensions may consume more memory than compressed file size suggests.
- **Frame/image processing:** Validate content types and resource limits for image transformations.
- **Regex DoS (ReDoS):** Poorly designed regular expressions can have excessive worst-case execution time.
- **Cache-poisoned DoS:** Cache behavior and request normalization can create availability problems; test only with approved, reversible cases.
- **HTTP header oversize:** Proxies and origin servers should enforce compatible limits.
- **Control characters:** Reject malformed header characters consistently.
- **Method override:** Allow only documented override behavior and apply the same authorization and resource controls.
- **Forwarded headers:** Trust only configured proxies and avoid letting arbitrary header values alter routing or cache behavior.
- **Source limitation:** The source includes payload-like examples and links, but does not establish that every technique works against a given service.

■■ Educational Checklists & Safe Examples

```
Target: [Your-Domain-or-Staging]
Feature: [Upload / Image / Form / Header / Cookie / Parser]
Approved maximum input: [Test-Plan-Limit]
Baseline latency: [Value]
Test latency/error rate: [Value]
Resource anomaly: [Yes/No]
Stop condition reached: [Yes/No]
Evidence: [Metrics and redacted logs]
```

- [ ] Request body, header, cookie, and field sizes are bounded.
- [ ] Uploads are checked for type, size, dimensions, and processing cost.
- [ ] Expensive operations have timeouts and concurrency limits.
- [ ] Regex and parsers have bounded worst-case behavior.
- [ ] Compression/decompression limits are enforced.
- [ ] Proxy and origin limits are compatible.
- [ ] Rate limits and monitoring are enabled.
- [ ] Stop conditions are defined before testing.

■■ Mitigation & Remediation

- Enforce limits for request bodies, headers, cookies, fields, uploads, image dimensions, and parser depth.
- Apply request timeouts, concurrency caps, queues, and resource budgets.
- Replace vulnerable regular expressions and use safe parsing libraries.
- Limit decompression ratios and reject malformed compressed input.
- Normalize proxy/cache behavior and keep infrastructure components patched.
- Rate-limit sensitive and expensive endpoints and monitor abnormal resource use.
- Return safe errors without leaking internal details.
- Run load and resilience tests in staging with a written plan and clear stop conditions.

■ Source Notes

Based on the supplied three-page *DOS.pdf*. Page 1 covers cookie bombs, long form inputs, image/frame processing, parameter changes, rate-limit behavior, ReDoS, cache-related DoS, and oversized HTTP headers. Page 2 discusses oversized headers, meta/control characters, method-override headers, and forwarding headers. The final page has limited readable text in the supplied preview. This rewrite converts the source into a controlled defensive testing guide and omits destructive payloads.

Educational use only. Availability testing requires explicit authorization and careful limits.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.